

Shubham Raut

PRN - 250840325067

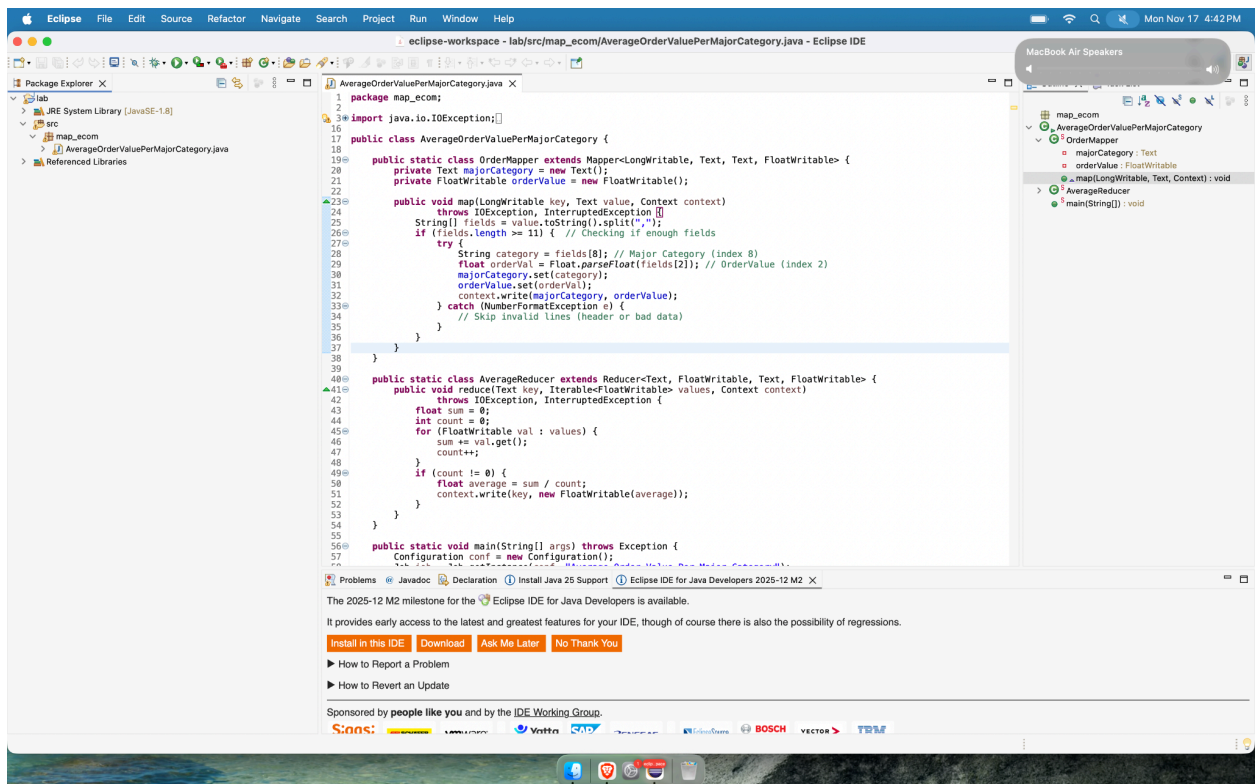
Submission: Provide screenshots and explanations for each step in a doc file. Doc file name 12 Digit PRN.

Question 1: MapReduce (5 Marks)

You are given the Java source code for a MapReduce program `AverageOrderValuePerMajorCategory.java`.

Tasks:

- **Compile the Java code and create a JAR file (one.jar).**



- **Upload the input file `ecom_data.csv`,, check if all classes are present in jar**

```

cdacoctnov0014@ip-172-31-15-133:~$ jar tvf one.jar
 25 Mon Nov 17 16:40:42 UTC 2025 META-INF/MANIFEST.MF
2517 Mon Nov 17 16:39:42 UTC 2025 map_ecom/AverageOrderValuePerMajorCategory$AverageReducer.class
2678 Mon Nov 17 16:39:42 UTC 2025 map_ecom/AverageOrderValuePerMajorCategory$OrderMapper.class
1843 Mon Nov 17 16:39:42 UTC 2025 map_ecom/AverageOrderValuePerMajorCategory.class
 576 Mon Nov 17 01:10:36 UTC 2025 .classpath
 362 Mon Nov 17 01:07:44 UTC 2025 .project
  55 Mon Nov 17 01:07:44 UTC 2025 .settings/org.eclipse.core.resources.prefs
 449 Mon Nov 17 01:07:48 UTC 2025 .settings/org.eclipse.jdt.core.prefs
cdacoctnov0014@ip-172-31-15-133:~$

```

- Run the MapReduce job using the compiled JAR.

```

Total time spent by all map tasks (ms)=2019
Total time spent by all reduce tasks (ms)=3594
Total vcore-milliseconds taken by all map tasks=4038
Total vcore-milliseconds taken by all reduce tasks=3594
Total megabyte-milliseconds taken by all map tasks=8269824
Total megabyte-milliseconds taken by all reduce tasks=14721024
Map-Reduce Framework
  Map input records=701
  Map output records=700
  Map output bytes=8537
  Map output materialized bytes=9949
  Input split bytes=117
  Combine input records=0
  Combine output records=0
  Reduce input groups=4
  Reduce shuffle bytes=9949
  Reduce input records=700
  Reduce output records=4
  Spilled Records=1400
  Shuffled Maps =2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=104
  CPU time spent (ms)=1610
  Physical memory (bytes) snapshot=1157451776
  Virtual memory (bytes) snapshot=11720978432
  Total committed heap usage (bytes)=1157627904
  Peak Map Physical memory (bytes)=729772032
  Peak Map Virtual memory (bytes)=2768646144
  Peak Reduce Physical memory (bytes)=214765568
  Peak Reduce Virtual memory (bytes)=4476452864
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=66706
File Output Format Counters
  Bytes Written=70

```

- cdacoctnov0014@ip-172-31-15-133:~\$ █

stored at

hadoop jar one.jar map_ecom/AverageOrderValuePerMajorCategory ecom_data.csv
/user/cdacoctnov0014/output/lab_result

- **Display** the output (part-r-00000) and (part-m-00000)from HDFS.

Home / user / cdacoctnov0014 / output / lab_result

☐	Name	Size	User	Group	Permissions	Date
☐	 .		cdacoctnov0014	cdacoctnov0014	drwxr-xr-x	November 17, 2025 04:36 AM
☐	 .		cdacoctnov0014	cdacoctnov0014	drwxr-xr-x	November 17, 2025 04:36 AM
☐	 _SUCCESS	0 bytes	cdacoctnov0014	cdacoctnov0014	-rw-r--r--	November 17, 2025 04:36 AM
☐	 part-r-00000	37 bytes	cdacoctnov0014	cdacoctnov0014	-rw-r--r--	November 17, 2025 04:36 AM
☐	 part-r-00001	33 bytes	cdacoctnov0014	cdacoctnov0014	-rw-r--r--	November 17, 2025 04:36 AM

Show 45 of 3 items Page 1 of 1

/ user / cdacoctnov0014 / output / lab_result / **part-r-00000**

```
Garden 31.958096
Household 30.592512
```

/ user / cdacoctnov0014 / output / lab_result / **part-r-00001**

```
Clothes 28.1271
Kitchen 37.25523
```

Question 2: Hive (18 Marks) (Retail data = ecom_data.csv)

Tasks:

1. **Create an external Hive table** for the Retail Sales dataset and load data into it. (3 marks)

```
hive> create external table retail_sales_ext (InvoiceNo string, UnitPrice FLOAT, OrderValue FLOAT, Quantity INT, Country string, InvoiceTime string, YearMonth string, MajorCategory string, MinorCategory string, Description string) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/user/cdacoctnov0014/retail_sales_ext';
OK
Time taken: 0.049 seconds
```

```
create external table retail_sales_ext (InvoiceNo string, UnitPrice FLOAT, OrderValue FLOAT,
Quantity INT, Country string, InvoiceDate string, InvoiceTime string, YearMonth string,
MajorCategory string, MinorCategory string, Description string)
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE
```

```
LOCATION '/user/cdacoctnov0014/retail_sales_ext';
```

```
LOAD DATA INPATH '/user/cdacoctnov0014/ecom_data.csv' into table retail_sales_ext;
```

2. Write a **query to find the maximum OrderValue** for each Major Category. (2 marks)

```
select MajorCategory, MAX(OrderValue) AS MaxOrderValue from retail_sales_ext group by
MajorCategory;
```

```
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 4
2025-11-17 13:00:48,537 Stage-1 map = 0%, reduce = 0%
2025-11-17 13:00:52,644 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.38 sec
2025-11-17 13:00:57,744 Stage-1 map = 100%, reduce = 25%, Cumulative CPU 3.08 sec
2025-11-17 13:00:58,763 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 8.15 sec
MapReduce Total cumulative CPU time: 8 seconds 150 msec
Ended Job = job_1762934466274_3921
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 4 Cumulative CPU: 8.15 sec HDFS Read: 103786 HDFS Write: 484 SUCCESS
Total MapReduce CPU Time Spent: 8 seconds 150 msec
OK
Clothes 488.16
Kitchen 510.0
Garden 633.6
Major Category NULL
Household 445.2
Time taken: 27.545 seconds, Fetched: 5 row(s)
hive> █
```

3. **Create a partitioned table** by Country and load the data accordingly. Also show the partitions created. (3 marks)

```
create table retail_sales_partitioned ( InvoiceNo string, UnitPrice FLOAT, OrderValue FLOAT,
Quantity INT, InvoiceDate string, InvoiceTime string, YearMonth string, MajorCategory string,
MinorCategory string, Description string )
```

```
PARTITIONED BY (Country string)
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE;
```

```
SET hive.exec.dynamic.partition=true;
```

```
SET hive.exec.dynamic.partition.mode=nonstrict;
```

```
insert into table retail_sales_partitioned PARTITION(Country)
```

```
select InvoiceNo, UnitPrice, OrderValue, Quantity, InvoiceDate, InvoiceTime, YearMonth,
MajorCategory, MinorCategory, Description, Country FROM retail_sales_ext;
```

```
SHOW PARTITIONS retail_sales_partitioned;
```

4. Write a query to **find the top 5 Minor Categories** with the highest number of orders using CTE. (2 marks)

```
with cnt AS (select MinorCategory, COUNT(*) as c from retail_sales_ext group by
MinorCategory)
```

```
select MinorCategory, c from cnt
```

```
order by c desc
```

```
limit 5;
```

5. **Create a managed table** copying data from the external table. (2 marks)

```
CREATE TABLE retail_sales_managed AS
```

```
SELECT * FROM retail_sales_ext;
```

6. Perform a **full outer join** between two copies of the table based on the InvoiceNo. (2 marks)

```
select a.*, b.*  
  
from retail_sales_managed a  
  
full outer join retail_sales_managed b  
  
ON a.InvoiceNo = b.InvoiceNo;
```

7. **Create a new managed table** retail_sales_orc stored as ORC format. (4 marks)
Insert data into this ORC table from the existing managed table retail_sales_managed.
Fetch data from both retail_sales_managed and retail_sales_orc.
Compare the query performance between the two tables and comment on any differences observed.

```
create table retail_sales_orc (InvoiceNo string, UnitPrice FLOAT, OrderValue FLOAT, Quantity  
INT, Country string, InvoiceDate string, InvoiceTime string, YearMonth string, MajorCategory  
string, MinorCategory string, Description string)
```

```
STORED AS ORC;
```

```
insert into table retail_sales_orc
```

```
select * from retail_sales_managed;
```

```
select * from retail_sales_managed;
```

```
select * from retail_sales_orc;
```

Question 3: PySpark RDD (12 Marks)

Load the RetailSales.csv file into an RDD. Split each line by commas.

```
>>> rdd = sc.textFile("ecom_data.csv")  
>>> header = rdd.first()  
>>> rdd = sc.textFile("/user/cdacoctnov0014/ecom_data.csv")  
>>> header = rdd.first()  
>>> header  
'InvoiceNo,UnitPrice,OrderValue,Quantity,Country,InvoiceDate,InvoiceTime,Year-Month,Major Category,Minor Category,Description'
```

1. Top Major Categories by Total Order Value (6 Marks)

Create an RDD of (Major Category, OrderValue) pairs.

Sum the OrderValue for each Major Category.

```
>>> data = rdd.filter(lambda line: line != header)
>>> split_rdd = data.map(lambda line: line.split(","))
>>> data.show(5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'PipelinedRDD' object has no attribute 'show'
>>> data.take(5)
[('549185', 0.85, 10.2, 12, United Kingdom, 7/4/11, 09:35, 2011-04, Clothes, Tops, PACK OF 20 NAPKINS PANTRY DESIGN', '576381', 2.95, 35.4, 12, United Kingdom, 15/11/11, 09:26, 2011-11, Clothes, Shoes, NATURAL SLATE HEART CHALKBOARD ', '551192', 1.25, 20.16, United Kingdom, 27/4/11, 10:54, 2011-04, Kitchen, Cutlery, 36 PENCILS TUBE SKULLS', '573553', 7.46, 7.46, 1, United Kingdom, 31/10/11, 13:48, 2011-10, Garden, Turf, SET 6 SCHOOL MILK BOTTLES IN CRATE', '539436', 2.51, 2.51, 1, United Kingdom, 17/12/10, 14:49, 2010-12, Garden, Hoses, FINE WICKER HEART ')]
>>> split_rdd.take(5)
[('549185', '0.85', '10.2', '12', 'United Kingdom', '7/4/11', '09:35', '2011-04', 'Clothes', 'Tops', 'PACK OF 20 NAPKINS PANTRY DESIGN', ['576381', '2.95', '35.4', '12', 'United Kingdom', '15/11/11', '09:26', '2011-11', 'Clothes', 'Shoes', 'NATURAL SLATE HEART CHALKBOARD '], ['551192', '1.25', '20', '16', 'United Kingdom', '27/4/11', '10:54', '2011-04', 'Kitchen', 'Cutlery', '36 PENCILS TUBE SKULLS'], ['573553', '7.46', '7.46', '1', 'United Kingdom', '31/10/11', '13:48', '2011-10', 'Garden', 'Turf', 'SET 6 SCHOOL MILK BOTTLES IN CRATE'], ['539436', '2.51', '2.51', '1', 'United Kingdom', '17/12/10', '14:49', '2010-12', 'Garden', 'Hoses', 'FINE WICKER HEART '])]
```

- Display the **top 5 Major Categories** with the **highest total Order Value**.

```
major_totals = major_rdd.reduceByKey(lambda a, b: a + b)
```

```
>>> top5_major = major_totals.takeOrdered(5, key=lambda x: -x[1])
```

```
top5_major = major_totals.takeOrdered(5, key=lambda x: -x[1])
```

```
>>> major_rdd.take(5)
[('Clothes', 10.2), ('Clothes', 35.4), ('Kitchen', 20.0), ('Garden', 7.46), ('Garden', 2.51)]
>>> major_totals.take(5)
[Stage 6:>
[Stage 7:>

[('Clothes', 4950.37), ('Kitchen', 6407.900000000001), ('Garden', 6040.080000000001), ('Household', 4986.579999999999)]
>>> major_totals.take(10)

^[[A
[Stage 17:>
                (0 + 0) / 1]
[Stage 17:>

[Stage 19:>
[Stage 19:>
                (0 + 1) / 1]

[('Clothes', 4950.37), ('Kitchen', 6407.900000000001), ('Garden', 6040.080000000001), ('Household', 4986.579999999999)]
>>>
```

2. Top Minor Categories by Product Count (6 Marks)

- Create an RDD of (Minor Category, 1) pairs.

```
>>> minor_rdd = split_rdd.map(lambda x: (x[9], 1))
>>> minor_rdd.take(5)
[('Tops', 1), ('Shoes', 1), ('Cutlery', 1), ('Turf', 1), ('Hoses', 1)]
>>> █
```

- **Count the total number of orders for each Minor Category.**

```
>>> top5_minor
[('Tops', 63), ('Shoes', 57), ('Hats', 56), ('Hoses', 51), ('Rakes', 51)]
```

- Display the **top 5 Minor Categories** with the **highest** number of orders.

```
>>> minor_counts = minor_rdd.reduceByKey(lambda a, b: a + b)
>>> minor_counts.take(5)
[('Tops', 63), ('Shoes', 57), ('Turf', 40), ('Hoses', 51), ('Cooking Knives', 25)]
>>>
```

all code:

```
rdd = sc.textFile("ecom_data.csv")
```

```
header = rdd.first()
```

```
data = rdd.filter(lambda line: line != header)
```

```
split_rdd = data.map(lambda line: line.split(","))
```

```
major_rdd = split_rdd.map(lambda x: (x[8], float(x[2])))
```

```
major_totals = major_rdd.reduceByKey(lambda a, b: a + b)
```

```
top5_major = major_totals.takeOrdered(5, key=lambda x: -x[1])
```

```
for x in top5_major:
```

```
    print(x)
```

```
minor_rdd = split_rdd.map(lambda x: (x[9], 1))
```

```
minor_counts = minor_rdd.reduceByKey(lambda a, b: a + b)
```

```
top5_minor = minor_counts.takeOrdered(5, key=lambda x: -x[1])
```

```
for x in top5_minor:
```

```
    print(x)
```

Question 4: Kafka (5 Marks)

Tasks:

- **Create a Kafka topic** named eg(retail_topic) with a replication factor of 1 and a single partition.

```
cdacoctnov0014@ip-172-31-15-133:~$ $KAFKA_HOME/bin/kafka-topics.sh --create --topic retail_topic097 --bootstrap-server master:9092
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic retail_topic097.
```

- **Start a Kafka consumer** that listens to the retail_topic and displays incoming messages in real-time.

```
cdacoctnov0014@ip-172-31-15-133:~$ $KAFKA_HOME/bin/kafka-console-consumer.sh --topic retailtopic097 --from-beginning --bootstrap-server master:9092
hello this
hellow
this is a test for the
lab
exam
```

- **Start a Kafka producer** that sends messages to the retail_topic.

```
>^Ccdacoctnov0014@ip-172-31-15-133:~$ $KAFKA_HOME/bin/kafka-console-producer.sh --topic retailtopic097 --bootstrap-server master:9092
>hello this
[2025-11-17 11:55:08,587] WARN [Producer clientId=console-producer] Error while fetching metadata with correlation id 4 : {retailtopic097=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
>hellow
>this is a test for the
>lab
>exam
```

- **Type any text into the producer console** — the same text should immediately appear on the consumer console in real-time.

```
>^Ccdacoctnov0014@ip-172-31-15-133:~$ $KAFKA_HOME/bin/kafka-console-producer.sh --topic retailtopic097 --bootstrap-server master:9092
>hello this
[2025-11-17 11:55:08,587] WARN [Producer clientId=console-producer] Error while fetching metadata with correlation id 4 : {retailtopic097=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
>hellow
>this is a test for the
>lab
>exam
hellow this is a realtime test
```

```
cdacoctnov0014@ip-172-31-15-133:~$ $KAFKA_HOME/bin/kafka-console-consumer.sh --topic retailtopic097 --from-beginning --bootstrap-server master:9092
hello this
hellow
this is a test for the
lab
exam
hello
this is a
realtime
test
hellow this is a realtime test
```